Лабораторная работа №9.
Работа с Modbus в STM32

Цель работы: научиться осуществлять передачу и приём данных по протоколу Modbus RTU, используя библиотеку FreeModbus и библиотеку HAL для STM32.

Введение в промышленные сети и протокол Modbus

Протокол Modbus — это открытый коммуникационный протокол, разработанный компанией Modicon в 1979 году. Благодаря своей простоте, открытости и надежности он стал одним из самых распространенных протоколов в области промышленной автоматизации по всему миру. Миллионы устройств, от датчиков и приводов до сложных ПЛК, используют Modbus для обмена данными.

Существует три основные модификации протокола

- **Modbus RTU (Remote Terminal Unit)** — использует двоичное кодирование и передается по последовательным интерфейсам (RS-485, RS-232). Наиболее популярная версия для промышленных сетей.
- **Modbus ASCII** — использует для передачи текстовые символы ASCII, менее эффективен, чем RTU, но удобен для отладки.
- **Modbus TCP** — адаптирует протокол для работы поверх сетей TCP/IP (Ethernet).

Данный материал ориентирован на вариацию **Modbus RTU**.

Теоретические основы Modbus RTU

Архитектура "Ведущий-Ведомый" (Master-Slave)

Modbus RTU использует архитектуру **"Ведущий-Ведомый"** (Master-Slave). В сети может быть только одно ведущее устройство (Master, также называемый клиентом) и до 247 ведомых устройств (Slave, или серверов)

Обмен данными всегда инициируется ведущим устройством. Ведомые устройства не могут начать передачу самостоятельно, они только отвечают на запросы мастера

Это простая, но надежная модель, исключающая конфликты в шине.

Физический уровень

Основным физическим интерфейсом для Modbus RTU является **RS-485**

Это дифференциальный последовательный интерфейс, который обеспечивает:

- передачу данных на расстояния до **1200 метров**;
- устойчивость к помехам;
- возможность подключения до 32 устройств на один сегмент без повторителей;
- скорости передачи от 1200 до 115200 бит/с и выше, при этом стандарт требует поддержки 9600 и 19200 бит/с.

Сеть строится на основе магистрального кабеля (витой пары) с обязательными терминальными резисторами на концах для подавления отражений сигнала

Структура данных в устройстве Slave

Данные в ведомом устройстве организованы в четыре основные таблицы. Это логические модели, а не физическая память.

Таблица1. Типы регистров в Modbus

Название	Тип данных	Доступ	Описание	Пример номера
Coils (Discrete Outputs)	1 бит	Чтение/Запись	Дискретные выходы (реле, флаги)	00001-09999
Discrete Inputs	1 бит	Только чтение	Дискретные входы (кнопки, датчики)	10001-19999
Input Registers	16 бит	Только чтение	Аналоговые входы, данные с датчиков	30001-39999
Holding Registers	16 бит	Чтение/Запись	Аналоговые выходы, настраиваемые параметры	40001-49999

Важно различать **номер регистра** (например, 40001) и его **адрес в сообщении** (0x0000). Разница между ними называется **смещением (offset)**

Например, регистр 40108 в сообщении будет иметь адрес 107 (0x006B), так как 40108 - 40001 = 107.

Структура сообщения Modbus RTU

Сообщение Modbus RTU состоит из следующих полей:

Таблица 2. Структура сообщения Modbus RTU

Поле	Размер	Описание
Адрес Slave (SlaveID)	1 байт	Адрес ведомого устройства (1-247). Адрес 0 используется для широковещательной рассылки.
Код функции	1 байт	Определяет действие, которое должен выполнить Slave (прочитать регистр, записать значение и т.д.).

Данные	N байт	Зависят от кода функции (адреса регистров, количество, значения для записи).
Контрольная сумма (CRC-16)	2 байта	Циклический избыточный код для проверки целостности сообщения.

Если отбросить адрес и контрольную сумму, оставшаяся часть называется **PDU (Protocol Data Unit)**.

Пример запроса: чтение 3 регистров Holding Registers, начиная с адреса 40108, от устройства с адресом 17 (0x11)

11 03 006B 0003 7687

- 11 - Адрес Slave.
- 03 - Код функции "Read Holding Registers".
- 006B - Адрес первого регистра (107 в HEX).
- 0003 - Количество регистров для чтения (3).
- 7687 - Контрольная сумма CRC-16.

Пример ответа:

11 03 06 AE41 5652 4340 49AD

- 11 - Адрес Slave.
- 03 - Код функции.
- 06 - Количество байт данных (6 байт для трёх 16-битных регистров).
- AE41 5652 4340 - Значения трёх регистров.
- 49AD - Контрольная сумма.

Основные коды функций

Таблица 3. Основные коды функций Modbus RTU

Код	**Название**	**Описание**	**Тип данных**
0x01	Read Coils	Чтение значений нескольких Coils	Битовый
0x02	Read Discrete Inputs	Чтение значений нескольких Discrete Inputs	Битовый
0x03	**Read Holding Registers**	**Чтение значений нескольких Holding Registers**	**16-битный**

0x04	Read Input Registers	Чтение значений нескольких Input Registers	16-битный
0x05	Write Single Coil	Запись одного значения Coil	Битовый
0x06	**Write Single Register**	**Запись одного значения Holding Register**	**16-битный**
0x0F	Write Multiple Coils	Запись нескольких значений Coils	Битовый
0x10	**Write Multiple Registers**	**Запись нескольких значений Holding Registers**	**16-бит**

Суть работы RS-485

Ключевая идея RS-485 — это **дифференциальная (сбалансированная) передача данных** по витой паре проводов. Это главное отличие от более простых интерфейсов вроде RS-232, которые используют несимметричные сигналы относительно общего провода.

1. Дифференциальный сигнал

- для передачи одного бита данных используются два провода: **A (Non-Inverting, "+")** и **B (Inverting, "-")**.
- логическому "0" (часто обозначается как *Space*) соответствует более высокое напряжение на линии A относительно линии B.
- логической "1" (часто обозначается как *Mark*) соответствует более низкое напряжение на линии A относительно линии B.
- например, при передаче "0": A = +2В, B = -2В. Разность потенциалов: +2В - (-2В) = +4В. При передаче "1": A = -2В, B = +2В. Разность потенциалов: -2В - (+2В) = -4В.

2. Преимущества дифференциальной передачи

- **Помехоустойчивость.** Любая электромагнитная помеха, наведенная на кабель, действует примерно одинаково на оба провода витой пары. Поэтому разность потенциалов между A и B почти не меняется. Приемник реагирует именно на эту разность, игнорируя общую помеху.
- **Дальность и скорость.** Благодаря помехоустойчивости, RS-485 позволяет передавать данные на значительные расстояния (до 1200 метров и более) на высоких скоростях (до 10 Мбит/с на коротких дистанциях).
- **Общая земля не критична.** Поскольку приемник считывает разность напряжений между A и B, точный потенциал "земли" приемника относительно "земли" передатчика не так важен. Это позволяет связывать устройства, находящиеся в разных зданиях с разными потенциалами земли.

3. Многопоточность (Multi-drop) и полудуплексный режим

- RS-485 — это интерфейс для **многоточечной** сети. К одной паре проводов (шине) можно подключить до **32** "стандартных" приемопередатчиков (transceivers). С использованием дополнительных повторителей (repeaters) и согласованных нагрузок это число можно увеличить до 256.
- Чаще всего используется **полудуплексный (half-duplex)** режим. Это означает, что устройства используют одну и ту же пару проводов для both передачи и приема. В один момент времени только **одно** устройство может передавать данные, остальные должны находиться в режиме приема. Управление режимом (передача/прием) осуществляется с помощью отдельного сигнала (например, DE /RE).

Физическое представление всех проводов RS-485

Физическая реализация зависит от режима работы (полудуплексный или полнодуплексный) и конкретной реализации устройства.

1. Полудуплексная 2-проводная

В этом режиме используется одна витая пара для обмена данными в обоих направлениях.

Таблица 4. Структура 2-проводной передачи в RS-485

Наименование провода/линии	**Обозначение**	**Описание и назначение**
Data A (или **D+**)	A	Неинвертирующий (+) дифференциальный сигнал.
Data B (или **D-**)	B	Инвертирующий (-) дифференциальный сигнал.
Экран (опционально, но рекомендуется)	SHIELD	Защитный экран кабеля. **Не является сигнальной землей!**

Как подключается. Все устройства подключаются к одной и той же паре A и B параллельно. На концах шины (у первого и последнего устройства) должны быть установлены **терминирующие резисторы** (согласующие), обычно номиналом 120 Ом, чтобы предотвратить отражение сигнала от концов линии.

Управление передачей. В каждом устройстве есть приемопередатчик с выводом для управления направлением (например, DE - Driver Enable, /RE - Receiver Enable). Чтобы начать передачу, устройство активирует свой передатчик (DE), тем самым "захватывая" шину.

2. Полнодуплексная 4-проводная шина

В этом режиме используются две отдельные витые пары: одна для передачи (TX), другая для приема (RX). Это позволяет передавать и принимать данные одновременно.

Таблица 5. Структура 4-проводной передачи в RS-485

Наименование провода/линии	**Обозначение**	**Описание и назначение**
Передача, A (TX+)	A_TX (или TXD+)	Неинвертирующий (+) сигнал для передачи.
Передача, B (TX-)	B_TX (или TXD-)	Инвертирующий (-) сигнал для передачи.
Прием, A (RX+)	A_RX (или RXD+)	Неинвертирующий (+) сигнал для приема.
Прием, B (RX-)	B_RX (или RXD-)	Инвертирующий (-) сигнал для приема.
Экран (опционально)	SHIELD	Защитный экран.

Задание:

1. Внедрите библиотеку FreeModbus в свой проект, в среде STM32CubeIDE.

   Ссылка для загрузки данной библиотеки представлена ниже:
   https://github.com/cwalter-at/freemodbus/archive/refs/heads/master.zip

   Также данную библиотеку можно загрузить из раздела ПО, на странице курса.

2. Проинициализируйте один таймер TIM
3. Реализуйте состыковку библиотеки FreeModbus и своего проекта, используя функции HAL для работы с UART TIM (в неблокирующем режиме) в следующих файлах:
   - port.h;
   - portserial.c;
   - porttimer.c.
4. Реализуйте Modbus Slave на своей плате STM32, с таблицей Holding Registers на 20 значений (с адреса 40001 по 40020).
   В каждый 5-й регистр таблицы (не выходя за рамки адресов 40001 по 40020), запишите следующие значения:
   - 1773;
   - 995;
   - 3981;
   - 451.

5. Установите на рабочий компьютер программу Modbus Poll.
   Ссылка для загрузки данной программы представлена ниже:
   https://www.modbustools.com/download/ModbusPollSetup64Bit.exe

   Также данную программу можно загрузить из раздела ПО, на странице курса.

6. Используя Modbus Poll в качестве Modbus-RTU Master, запросите у Вашего Modbus Slave 20 регистров, начиная с адреса 40001.
   Результат запроса представьте в виде скриншота Вашей таблицы в программе Modbus Poll.

7. *Необязательно. Реализуйте через Modbus Poll запись ячеек, которые не были изменены на шаге 4, следующими значениями: 20; 30; 40; 50; 60; 80; 90; 100; 120; 140; 160; 180; 200; 220; 240; 260

   Примечание: для записи одного регистра используется команда **Write Single Register (код 0x06).**

Результат записи представьте в виде скриншота отладки Вашего проекта, где лежат записанные значения в Вашем массиве таблицы Holding Registers.

8. Результаты проделанных шагов 1-7 (опционально 8) представьте в файле отчёта. Также приложите в ответе Ваш проект.